UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ing. En Tecnologías de la Información e Innovación Digital

Fundamentos de Inteligencia Artificial

2.1. Problemas de búsqueda
2.2. Espacios de búsqueda

Bohm, cap. 1-2

NOMBRE DEL ESTUDIANTE:

Jorge Lidio Barragan Villanueva

NOMBRE DEL DOCENTE:

Mtro. Javier García Lara

GRUPO:

172

2.1 Problemas de búsqueda

2.2 Espacios de búsqueda Bohm, cap. 1-2

Modelado + Código: Modelar 2 problemas reales como problemas de búsqueda (estado, acciones, coste, meta). Representar cada espacio como grafo (dirigido/ponderado). Programar en Python un generador de estados y un grafo con NetworkX. Entregar diagrama del grafo, código comentado y análisis de complejidad.

Traslado en un campus universitario

Un auxiliar debe trasladar un equipo desde la entrada al laboratorio CADCAM. Hay rutas con diferentes tiempos y sentidos de circulación. Se busca minimizar el tiempo total de traslado. El nodo L representa la entrega terminada.

ID	Lugar	ID	Lugar
E	Entrada	A	Administración
B	Biblioteca	P	Patio
C	Cómputo	R	Rampa
T	Taller	L	Laboratorio CADCAM

Datos: arcos dirigidos y tiempos

Origen	Destinos permitidos (min)
E	A: 3; B: 2; L: 20
A	P: 4; C: 6
B	P: 2; R: 5
P	C: 3; T: 4; A: 4
R	T: 2; C: 3
C	L: 4
T	L: 2
L	Sin sucesores: la entrega está terminada.

Estado, acciones, transición, coste y meta

Estado: ubicación actual $s \in \{E, A, B, P, C, R, T, L\}$. **Inicial:** E. **Acciones:** recorrer un arco saliente autorizado. **Transición:** $T(u, \text{ir a } v) = v$. **Coste:** minutos del arco $u \rightarrow v$. **Meta:** $s = L$. **Objetivo:** minimizar $g = \Sigma\, c(u,v)$.

El estado no necesita recordar todo el camino: bajo los supuestos, las opciones futuras dependen solo de la ubicación actual. Los arcos A → P y P → A forman un ciclo; el conjunto de estados vistos evita expandirlo indefinidamente.

Menos minutos, más de un tramo

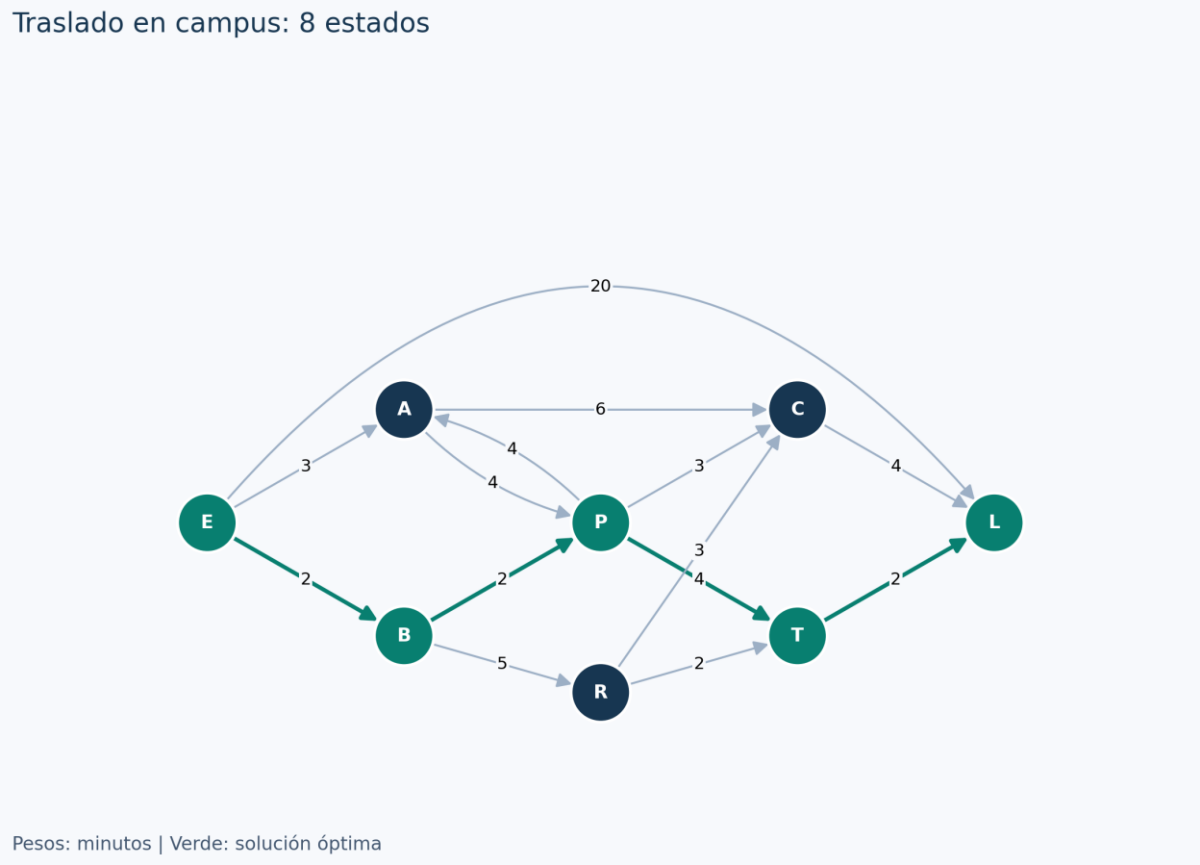


Ilustración 1 - Grafo dirigido y ponderado: 8 estados y 14 arcos. Las flechas indican acciones permitidas y sus etiquetas, minutos. El camino óptimo está destacado en verde.

Solución: E → B → P → T → L.
Coste mínimo: 2 + 2 + 4 + 2 = **10 minutos**.

Procedimiento y contraste

El generador produce los sucesores definidos por el mapa; la expansión construye todos los estados alcanzables. Dijkstra compara costes acumulados y devuelve la ruta mostrada. La comprobación exhaustiva enumera las **9 rutas simples** de E a L y confirma que ninguna tiene coste inferior.

El arco directo E → L usa una sola acción y cuesta 20 minutos. La solución mínima usa cuatro acciones y cuesta 10. Con costes positivos, repetir un ciclo aumenta el coste, por lo que es suficiente contrastar caminos simples. El resultado es óptimo para los datos del modelo, no una recomendación de ruta real dentro de UPA.

Programación de mantenimiento

Un técnico realizará cuatro tareas sobre un equipo fuera de servicio. Las tres primeras pueden ordenarse libremente; la validación requiere completarlas todas. Cambiar de tarea consume tiempo de preparación. Se minimiza la duración total de una secuencia sin tareas simultáneas.

ID	Actividad	Duración	Requisito
I	Inspeccionar el equipo	4 min	Ninguno
L	Limpiar filtro / ventilación	6 min	Ninguno
C	Revisar cableado	3 min	Ninguno
V	Validar funcionamiento	5 min	I, L y C completadas

Preparación para cambiar de tarea (minutos)

Ultima / próxima	I	L	C	V
O: inicio	2	4	1	-
I	-	1	2	3
L	3	-	4	2
C	3	3	-	1

Un guion indica una transición no aplicable. V solo está habilitada si I, L y C ya terminaron. El programa guarda ceros en entradas que nunca se usan.

Formulación del estado

Estado: s = (M, u), donde M es el conjunto de tareas hechas y u es la última tarea; O indica el inicio. **Inicial:** (∅, O). **Acción:** ejecutar una tarea t pendiente y con requisitos cumplidos. **Transición:** (M,u) → (M ∪ {t},t). **Coste:** duración(t) + preparación(u,t). **Meta:** ({I,L,C,V},V). **Objetivo:** minimizar la suma de tiempos.

Por qué guardar la última tarea: tener hechas I y C puede significar haber terminado en I o en C. Preparar L cuesta 1 o 3 minutos respectivamente; fusionar ambos estados produciría costes futuros incorrectos.

Implementación: máscara binaria I=1, L=2, C=4 y V=8. Por ejemplo, M=5 representa {I,C}. El estado (5,I) tiene la misma información que ({I,C},I).

Un grafo de decisiones parciales

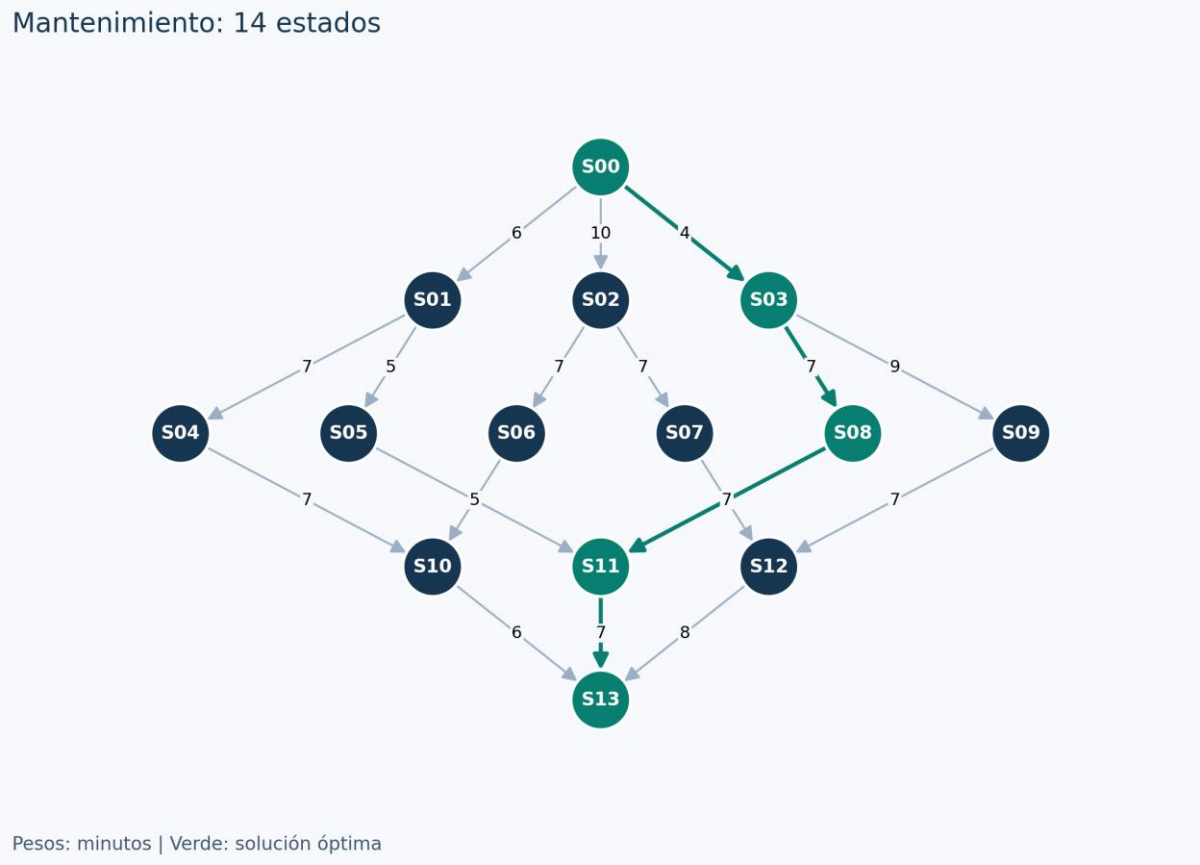


Ilustración 2 - Grafo dirigido acíclico y ponderado: 14 estados y 18 arcos. Las capas agrupan estados con 0, 1, 2, 3 y 4 tareas completadas. Los ID se definen en la tabla de la página siguiente.

Camino óptimo: S00 → S03 → S08 → S11 → S13.
Orden: C → I → L → V. **Tiempo:** 4 + 7 + 7 + 7 = **25 minutos**.

Generación y fusión de estados

El generador revisa cada tarea: descarta las ya hechas y las que tienen requisitos pendientes. Cada arco añade una tarea, por lo que no hay ciclos. Se fusionan historias diferentes solo cuando coinciden el conjunto de tareas completadas y la última tarea. Por ejemplo, I → C → L y C → I → L llegan a S11; Dijkstra conserva el menor coste de llegar a ese mismo estado.

Que existan 6 órdenes completos no significa que haya 6 estados: los estados también representan decisiones parciales. El grafo conserva la información necesaria sin guardar el historial completo de cada secuencia.

Interpretación de los resultados

ID	Máscara	Tareas completadas M	Ultima u
S00	0	∅	O
S01	1	{I}	I
S02	2	{L}	L
S03	4	{C}	C
S04	3	{I,L}	L
S05	5	{I,C}	C
S06	3	{I,L}	I
S07	6	{L,C}	C
S08	5	{I,C}	I
S09	6	{L,C}	L
S10	7	{I,L,C}	C
S11	7	{I,L,C}	L
S12	7	{I,L,C}	I
S13	15	{I,L,C,V}	V

Comprobación exhaustiva independiente

Orden	Minutos	Orden	Minutos
I → L → C → V	26	L → C → I → V	32
I → C → L → V	27	C → I → L → V	25
L → I → C → V	28	C → L → I → V	28

Los seis órdenes cumplen la precedencia de V. El mínimo es único en esta instancia: C → I → L → V. Sus tareas suman 18 minutos y la preparación 1 + 3 + 1 + 2 = 7 minutos, para un total de 25.

Coste de representar y buscar

Sea V el número de estados alcanzables y E el de transiciones. Los costes siguientes corresponden a las estructuras usadas; almacenar y dibujar todo el grafo es una decisión de esta práctica, no un requisito de todos los algoritmos de búsqueda.

Operación	Tiempo	Memoria / explicación
Generar sucesores de ruta	O(grado de salida)	O(1) auxiliar; yield entrega un sucesor cada vez.
Construir grafo de ruta	O(V + E)	O(V + E), incluyendo grafo, cola y estados vistos.
Dijkstra con heap binario	O((V + E) log V)	O(V + E) como cota de almacenamiento de esta implementación, incluido el grafo.
Construir grafo de tareas	O(Vn + E)	Se revisan n tareas por estado; el grafo ocupa O(V + E).
Enumeración de verificación	Solo instancia pequeña	No se propone como método escalable para resolver el problema.

Crecimiento del problema de mantenimiento

Con n tareas, representar un estado por (M,u) produce como cota $\mathbf{1 + n \cdot 2^{n-1}}$ estados: por cada tarea u, hay 2^{n-1} subconjuntos que la contienen. Las precedencias reducen los estados alcanzables. Si cada estado permite a lo sumo n acciones, $E \leq nV$. Por tanto, materializar este modelo tiene crecimiento exponencial en n, aunque Dijkstra sea polinómico en el tamaño del grafo explícito.

Para n=4, la cota de estados es 33, pero solo se alcanzan 14; la restricción que sitúa V al final elimina combinaciones. Se guardan 18 transiciones. Las máscaras de estas instancias caben en una palabra: se consideran O(1) sus operaciones. Para n muy grande, copiar, comparar y aplicar operaciones a máscaras o conjuntos añade un coste ligado a su longitud.

Completitud, optimalidad y alternativas

En estos grafos finitos, Dijkstra encuentra una solución si existe y devuelve el menor coste con los pesos no negativos. El conjunto de vistos de la construcción no fija los costes: Dijkstra vuelve a considerar mejoras sobre el grafo ya construido. Para el segundo problema, su estructura acíclica también permite resolver caminos mínimos mediante un orden topológico en O(V + E).

Conclusión: elegir un buen estado determina la corrección del modelo; elegir un algoritmo determina cómo se explora. Minimizar minutos exige respetar pesos, precedencias e información relevante del estado.

Referencias bibliográficas

Bohm, R. A. (Ed.). (2021). *The fundamentals of search algorithms*. Nova Science Publishers. https://novapublishers.com/shop/the-fundamentals-of-search-algorithms/

GitHub. (s. f.)yiHubopps. GitHub Docs.

https://github.com/yiHubopps/Fundamentos-de-IA

NetworkX Developers. (s. f.). *Shortest paths*. NetworkX documentation. Recuperado el 18 de octubre de 2026, de https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html

NetworkX Developers. (s. f.). *single_source_dijkstra*. NetworkX documentation. Recuperado el 18 de octubre de 2026, de https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.shortest_paths.weighted.single_source_dijkstra.html

Python Software Foundation. (s. f.). *collections: Container datatypes*. Python documentation. Recuperado el 18 de octubre de 2026, de https://docs.python.org/3/library/collections.html

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson. https://aima.cs.berkeley.edu/algorithms.pdf